

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Part D | Abstract, Objectives, Novelty, Architecture, Dataset and AWS Services

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Pratyush Chandrasekhar (24BIT0226) | Subhrojyoti Das (24BIT0194) | Mehul Anand (24BIT0185)

Repository: github.com/SUBHRO71/SmartStorageTierRecommenderForMedicalSector_Cloud_Project_2026

D1. Abstract

Hospital imaging archives now grow by terabytes every year, and the cost of keeping that data is one of the largest recurring line items in a radiology department's IT budget. Storing every scan on high-performance cloud storage is financially unsustainable, but moving everything to cheap archival storage is clinically unsafe: retrieving a study from Amazon S3 Glacier Deep Archive can take up to twelve hours, and a radiologist comparing a follow-up scan against a prior cannot wait that long.

Existing solutions fail from opposite directions. Cloud storage tiering, including AWS's own S3 Intelligent-Tiering and the learning-based systems in the recent literature, decides where a file belongs using only block-level behaviour such as object size, age and access frequency. It is completely blind to what the file contains. Medical deep learning, meanwhile, has become very good at reading scans, but its output is used only to assist the radiologist and is never fed back to the storage layer. Two identical-looking files, one a routine normal chest X-ray and the other an active oncology follow-up, are treated identically today.

This project closes that gap. A deep learning model reads each scan and, together with its DICOM metadata and access history, predicts the probability that the study will be retrieved again. An explicit cost rule then converts that probability into one of four Amazon S3 storage classes by minimising expected total cost, deliberately weighting a wrongly archived scan far more heavily than a marginally higher storage bill. The decision is written as an S3 object tag and enforced by a tag-filtered Lifecycle rule, so the model sets policy while AWS performs the mechanism.

The system is deployed entirely serverless on Amazon S3, AWS Lambda, Amazon DynamoDB, Amazon API Gateway, Amazon Cognito, Amazon CloudFront, Amazon CloudWatch, Amazon SNS, AWS IAM and AWS Budgets, so idle cost is zero and the whole system fits inside always-free AWS allowances.

Word count: approximately 290.

D2. Project Objectives

Each objective below is stated so that it can be measured and either met or missed. Targets are first-pass goals and will be reported honestly against in Phase-II, including where they are not met.

- 1. Build a deep learning tier recommender.** Train a model that predicts, for each individual scan, the probability that it will be retrieved again, using image-derived features together with DICOM metadata and access history. Target: ROC-AUC of at least 0.75 on a patient-disjoint test split, with a calibration curve reported alongside, since the cost rule consumes the probability directly.
- 2. Convert probability into a storage class using an explicit, auditable cost rule.** Select from S3 Standard, Standard-IA, Glacier Flexible Retrieval and Glacier Deep Archive by minimising expected total cost. The rule must account for storage, request, transition and retrieval charges, minimum storage durations, the 128 KB minimum billable object size and the 40 KB per-object Glacier metadata overhead.

3. **Reduce projected storage cost measurably.** Achieve at least a 40 percent reduction in projected monthly storage cost against an all-S3-Standard baseline, and outperform both a conventional age-based lifecycle rule and S3 Intelligent-Tiering on the same 112,120-record dataset.
4. **Keep clinically unsafe placements rare.** Hold the wrongly-archived rate, defined as the proportion of scans that were subsequently needed but had been placed in Glacier or Deep Archive, below 5 percent. This metric is reported next to the cost saving in every result, never separately.
5. **Deploy a working serverless system on AWS within the free tier.** Demonstrate an end-to-end path from scan upload through automated tier decision to a real S3 Lifecycle transition, using no EC2 or RDS, with total AWS spend below one US dollar.
6. **Deliver an authenticated dashboard that explains its decisions.** Provide a Cognito-authenticated web dashboard showing tier distribution, projected cost, and for any individual scan the predicted probability and the reason it was assigned its storage class.

D3. Novelty Summary

Across the fifteen papers surveyed by this team, no existing work uses the content of a medical image, together with its metadata and access history, to decide which cloud storage class that image should occupy. The novelty of this project is therefore not an incremental improvement to an existing method; it is the connection of two mature fields that have not previously been joined.

New feature | content-aware storage tiering

Every tiering system in the surveyed literature judges a file by behaviour alone. Sibyl, for example, describes each object with exactly six numbers: request size, request type, access interval, access count, free capacity and current placement. Not one of them describes what the data is. A hospital already knows the modality, body part, patient age, study description and reported findings at the moment a scan is created, all of it free in the DICOM header. This project is the first to feed that information, plus a learned representation of the image itself, into the placement decision.

Better algorithm | asymmetric, two-stage decision

Rather than a four-class classifier, which would treat every misclassification as equally bad, the system separates a learned probability from an explicit cost rule. This makes each decision explainable, allows the clinical penalty to be re-tuned and every result regenerated without retraining, and encodes in inspectable code the fact that stranding a needed scan in Deep Archive is a far worse error than paying slightly more for fast storage.

Better architecture | policy separated from mechanism

The model does not move data. It writes an S3 object tag, and a tag-filtered Lifecycle rule performs the transition. AWS handles retries and auditing, the decision is visible in the console, and a failure in the model can never leave an object half-migrated.

Better AWS integration | a cost model that is actually correct

The closest prior work, Khan et al. (2024), explicitly omitted inter-tier transfer fees and early-deletion penalties, and our own gap analysis identified this as a limitation. This project models the full charge structure, including the 128 KB minimum billable object size and the 40 KB per-object Glacier metadata overhead, which are invisible at the headline per-GB rate but materially change the economics of archiving medical images. Reported savings are therefore conservative and defensible rather than optimistic.

Better security and automation

Amazon Cognito authenticates users and API Gateway validates the resulting tokens, while AWS IAM grants each component only the permissions it needs. The pipeline is fully event-driven: a scan upload triggers the decision automatically, with no scheduled job and no human in the loop, and Amazon SNS raises an alert whenever a scan is retrieved from archive so that mistakes surface immediately rather than being discovered by a waiting clinician.

Better scalability

Being entirely serverless, the system scales from a single scan to an entire hospital archive without provisioning, and costs nothing when idle. Inference runs in Lambda in milliseconds because the expensive image-embedding step is precomputed offline rather than repeated per request.

D4. Proposed Architecture

Two diagrams are provided, as required. Figure 1 shows how the AWS services interact, covering data flow, storage, processing, authentication, notifications and monitoring. Figure 2 shows the overall project workflow from dataset through to evaluation. Both appear on the following landscape pages.

Both are generated by `architecture/make_diagrams.py` in the repository, so they can be regenerated whenever the design changes rather than being redrawn by hand. The Mermaid sources in `architecture/README.md` remain the master reference, and the reasoning behind each design choice, including the alternatives rejected, is recorded as eight architecture decision records in `architecture/decisions.md`.

Diagram 1 | AWS Cloud Architecture

AWS Cloud Architecture

Smart Storage Tier Recommender for Medical Image Repositories | BCSE355L Project Phase-I | Region: us-east-1

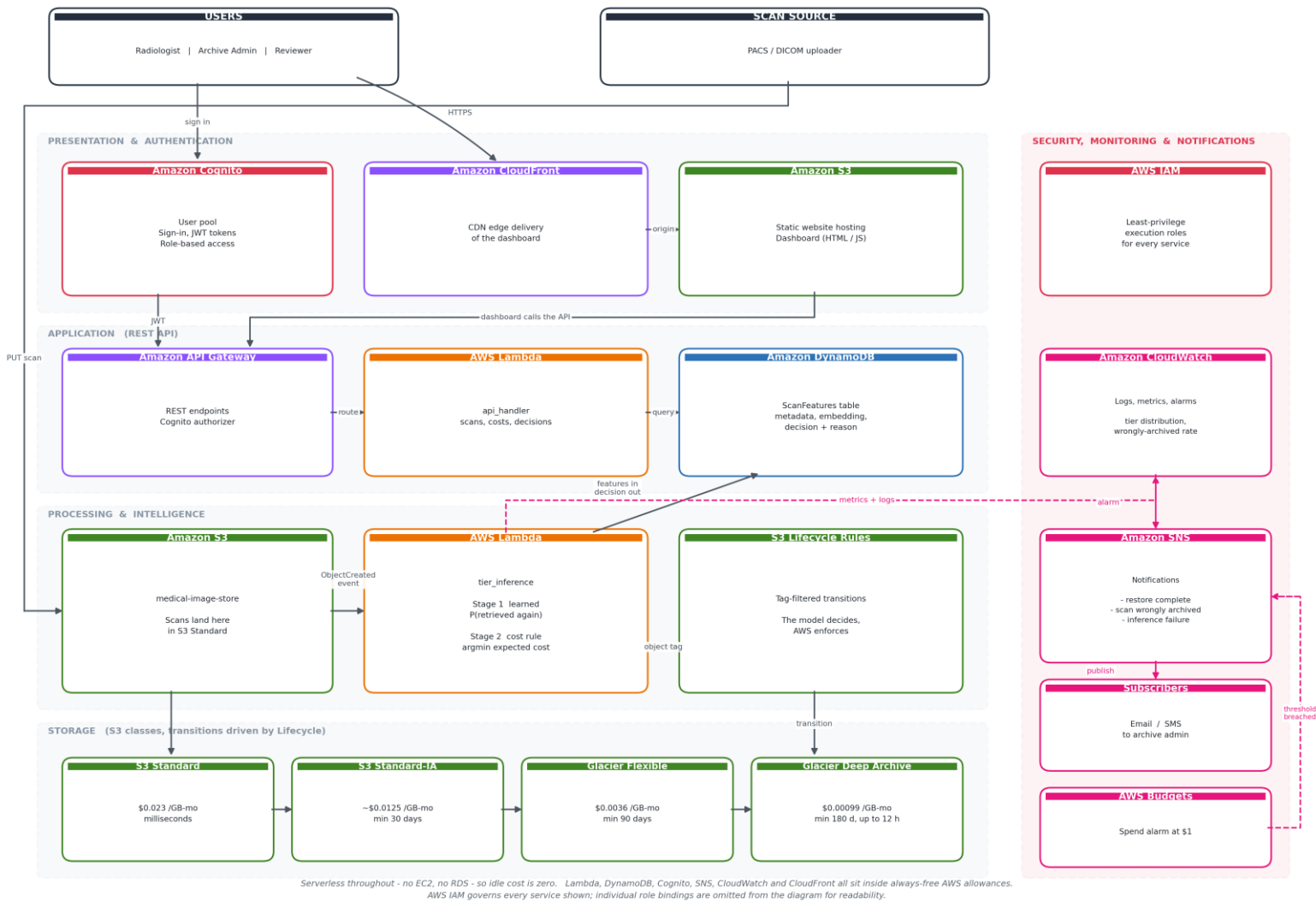


Figure 1. AWS Cloud Architecture. Four layers - presentation and authentication, application, processing and intelligence, and storage - with a cross-cutting security, monitoring and notification column on the right.

Diagram 2 | Complete System Architecture

Complete System Architecture

End-to-end workflow: offline training, online tier decision, feedback loop and cost evaluation



Figure 2. Complete System Architecture. The offline training pipeline, the per-scan online decision, the feedback loop from retrieval events, and the offline cost evaluation against three baselines.

D4.1 How the AWS services interact (Figure 1)

The architecture is arranged in four horizontal layers with a cross-cutting security, monitoring and notification column on the right.

- **Authentication.** A radiologist or archive administrator signs in through an Amazon Cognito user pool, which issues a JWT. Amazon API Gateway is configured with a Cognito authorizer, so every API call is validated before it reaches any Lambda function.
- **Presentation.** The dashboard is a static site held in Amazon S3 and delivered through Amazon CloudFront. It holds no credentials and never contacts AWS services directly; it calls the REST API only.
- **Application.** Amazon API Gateway routes requests to an `api_handler` Lambda, which reads and writes an Amazon DynamoDB table holding each scan's metadata, image embedding, predicted class and the reason for that decision.
- **Data flow and processing.** A new scan is uploaded to the `medical-image-store` S3 bucket and lands in S3 Standard. The resulting `ObjectCreated` event triggers the `tier_inference` Lambda, which reads the scan's features from DynamoDB, runs the Stage 1 model to obtain a retrieval probability, applies the Stage 2 cost rule, and writes the chosen class back as an S3 object tag.
- **Storage.** A tag-filtered S3 Lifecycle rule performs the actual transition between S3 Standard, Standard-IA, Glacier Flexible Retrieval and Glacier Deep Archive. Lifecycle evaluates once daily, so transitions are not instantaneous.
- **Monitoring.** Amazon CloudWatch collects logs and custom metrics, notably the distribution of scans across classes and the wrongly-archived rate, and raises alarms on threshold breaches.
- **Notifications.** Amazon SNS publishes to email and SMS subscribers when an archive restore completes, when a scan is retrieved from cold storage and therefore logged as wrongly archived, and when inference fails. AWS Budgets publishes through the same topic if spend crosses one dollar.
- **Security.** AWS IAM provides least-privilege execution roles for every component. The inference function can tag objects in one bucket and write to one table, and can do nothing else.

D4.2 Overall system workflow (Figure 2)

The workflow runs in four stages. Stage 1 is offline and free: the ChestX-ray14 dataset is parsed, the retrieval label is derived, MobileNetV3 produces a 128-dimensional embedding per image on a free Colab GPU, and the Stage 1 probability model is trained. The resulting model artifact is deployed to Lambda and the embeddings are loaded into DynamoDB.

Stage 2 is the online per-scan decision already described. Stage 3 is the feedback loop: when a doctor requests a scan, a fast-tier hit is served in milliseconds, while an archive miss incurs a restore job of several hours, is logged as wrongly archived and raises an SNS alert. Either way the access counters in DynamoDB are updated and feed the next decision for that scan.

Stage 4 is evaluation, performed offline across all 112,120 records so that no AWS cost is incurred. The full cost model is applied to our policy and to three baselines, and the monthly bill is always reported together with the wrongly-archived rate.

Note on scale

The mechanism is demonstrated live on approximately 1,000 real images, which fits inside the 5 GB always-free S3 allowance, while cost is projected across all 112,120 records using AWS's published price list. This is stated explicitly rather than presented as a full-scale measurement. The same approach was taken by Khan et al. (2024), who used Google Cloud prices on simulated data; this project improves on it by including the transition and early-deletion costs that paper omitted.

D5. Dataset Details

Attribute	Detail
Dataset Name	NIH ChestX-ray14 (also published as ChestX-ray8)
Source	National Institutes of Health Clinical Center, USA. Released with Wang et al., "ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases", CVPR 2017.
URL	https://nihcc.app.box.com/v/ChestXray-NIHCC Mirrors: Kaggle (nih-chest-xrays/data) and Academic Torrents
Size	Approximately 45 GB in total, distributed as 12 compressed archives. Average image size approximately 400 KB. A working subset of roughly 1,000 images (about 400 MB) is used for the live AWS demonstration.
Number of Records	112,120 frontal-view chest radiographs from 30,805 unique patients. Mean of approximately 3.6 images per patient.
Number of Features	Raw metadata: 11 columns in Data_Entry_2017.csv (Image Index, Finding Labels, Follow-up #, Patient ID, Patient Age, Patient Gender, View Position, Original Image Width and Height, and Original Image Pixel Spacing x and y). Engineered feature vector: 140 dimensions, comprising a 128-dimensional MobileNetV3 image embedding plus approximately 12 metadata and derived features (patient age, gender, view position, follow-up index, prior study count, encoded finding labels, object size in bytes).
Data Type	Images: PNG, 1024 x 1024 pixels, 8-bit greyscale. Metadata: CSV, mixed categorical and numeric. Derived label: binary.
License	Publicly released by the NIH Clinical Center for research and educational use. No registration, credentialing or data use agreement is required. Attribution to Wang et al. (CVPR 2017) is required in any publication. The images are de-identified; no protected health information is present.
Purpose of using the dataset	Two purposes. First, to derive the retrieval-likelihood label that the Stage 1 model learns, using the Follow-up # column. Second, to supply the image content and metadata features that make this project content-aware rather than behaviour-only. It was selected specifically because it requires no clearance of any kind, which suits an undergraduate team with no institutional data access.
Preprocessing required	1. Parse Data_Entry_2017.csv and group rows by Patient ID. 2. Derive the binary label: a scan is labelled 1 if its Follow-up # is below the maximum for that patient, indicating a later study exists and the earlier one was very likely retrieved for comparison. 3. Split by Patient ID, never by image, so that scans of the same patient cannot appear in both training and test sets. 4. Resize to 224 x 224 and normalise for the embedding network. 5. Extract the 128-dimensional embedding for every image. 6. Encode categorical metadata; derive prior-study counts and approximate intervals from Patient Age deltas. 7. Address class imbalance and record the fixed random seed for reproducibility.

Known limitations of this dataset, stated openly

- No timestamps. ChestX-ray14 records follow-up ordering but not dates, so intervals are approximated from Patient Age deltas and are coarse.
- The label is a proxy, not ground truth. It cannot capture retrievals that occur without a subsequent scan, such as multidisciplinary team meetings, second opinions or research requests. A sensitivity analysis on this assumption is planned.
- Images are distributed as PNG rather than DICOM, so genuine DICOM header fields are simulated from the available metadata columns.
- Chest radiography only. Findings will not necessarily generalise to CT or MRI, whose objects are far larger and whose retrieval patterns differ.

Why no public dataset can supply the true label

No publicly available dataset records who retrieved which scan and when. Those records exist only inside hospital PACS audit trails in IHE ATNA format, are protected health information, and are never released. The training label therefore cannot be downloaded and must be derived. There is published precedent for exactly this proxy: Yayan et al., Scientific Reports, December 2025, on metadata-derived proxies of 90-day follow-up in the same 112,120 ChestX-ray14 radiographs.

D6. AWS Services Planning

AWS Service	Purpose in this project	Free-tier position	Owner
Amazon S3	Stores the medical images; also hosts the static dashboard. The four storage classes are what the model predicts.	5 GB Standard, 20,000 GET, 2,000 PUT per month	Mehul, Pratyush
AWS Lambda	tier_inference runs the two-stage decision on upload; api_handler serves the REST API.	1M requests and 400,000 GB-seconds per month, always free	Mehul, Subhrojyoti
Amazon DynamoDB	Stores per-scan metadata, the image embedding, access history and the recorded decision with its reason.	25 GB and 25 RCU/WCU, always free	Subhrojyoti
Amazon API Gateway	Exposes the REST API to the dashboard, with a Cognito authorizer validating every request.	Consumes credits beyond the free allowance	Subhrojyoti
Amazon Cognito	Authenticates radiologists and archive administrators; issues JWTs and enforces role-based access.	Generous monthly active user allowance	Subhrojyoti
Amazon CloudFront	Delivers the dashboard from edge locations.	Always-free monthly allowance	Pratyush
Amazon CloudWatch	Logs, custom metrics such as tier distribution and wrongly-archived rate, dashboards and alarms.	10 custom metrics and 5 GB of log ingestion free	Mehul, Pratyush
Amazon SNS	Notifies on archive restore completion, wrongly archived scans, inference failures and budget threshold breaches.	1M publishes and 1,000 email notifications free	Mehul
AWS IAM	Least-privilege execution roles and policies for every component.	Free	Subhrojyoti
AWS Budgets	Spend alarm set at one dollar, configured before any other resource is created.	Free	All members

Services deliberately not used

Amazon EC2 and Amazon RDS are excluded by design. AWS replaced the Free Tier for new accounts on 15 July 2025; accounts created after that date receive credits on a six-month clock rather than twelve months of free EC2. Lambda, DynamoDB, Cognito, SNS and CloudFront retain always-free monthly allowances that neither consume credits nor expire. A fully serverless design therefore has zero idle cost, and no forgotten instance can drain the budget. Amazon SageMaker is also excluded: model training runs free on Google Colab, and inference is small enough to run inside Lambda.

D7. Implementation Status and Repository

Deliverable	Status
Literature survey, 15 papers across the team	Complete
Individual research gap analyses	Complete, Parts A to C of this document
Abstract, objectives, novelty	Complete
AWS Cloud Architecture diagram	Complete, Figure 1
Complete System Architecture diagram	Complete, Figure 2
Dataset selection and documentation	Complete, NIH ChestX-ray14
AWS services plan	Complete
GitHub repository, branches and pull requests	Active
Preprocessing and label derivation	Not started, Phase-II
Stage 1 model training	Not started, Phase-II
Cost model and baseline comparison	Not started, Phase-II
AWS deployment	Not started, Phase-II

Repository

github.com/SUBHRO71/SmartStorageTierRecommenderForMedicalSector_Cloud_Project_2026

Branching follows main, develop and one feature branch per member. The architecture design and this documentation were merged through pull request #1. Architecture decisions, including the alternatives that were rejected and why, are recorded as eight ADRs in `architecture/decisions.md`.

Immediate next steps for Phase-II

7. Confirm the AWS account creation date, which determines the applicable free-tier regime.
8. Create the AWS Budgets alarm at one dollar before provisioning any other resource.
9. Download the ChestX-ray14 subset, derive the label and build patient-disjoint splits.
10. Train and calibrate the Stage 1 probability model.
11. Implement the full cost model and compare against the three baselines.
12. Deploy the serverless stack and demonstrate a real Lifecycle transition.